



MAKERERE

UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

CSC 2207 ROBOTICS

Forest Firefighting Autonomous Drone

Practical Simulation Report

TUSIIME MARK
24/U/11684/PS
2400711684

Project Repository: <https://github.com/tusiim3/Forest-Firefighting-Autonomous-Drone>

1. System Architecture

The system uses 2 single Mavic 2 Pro drones running inside the Webots simulator. My controller is implemented in one Python file (`autonomous_mavic.py`), which handles flight stabilisation, waypoint patrol, smoke detection, visual tracking and water dropping. The provided supervisor (`fire.py`) was left unchanged and is responsible for creating fires, spreading them and responding to water-drop commands sent through the `customData` channel.

The drone uses a downward-facing camera, GPS, IMU and gyroscope. During every simulation step I read these sensors, update the drone's position and attitude, detect smoke if present, decide what behaviour to perform, then compute motor commands.

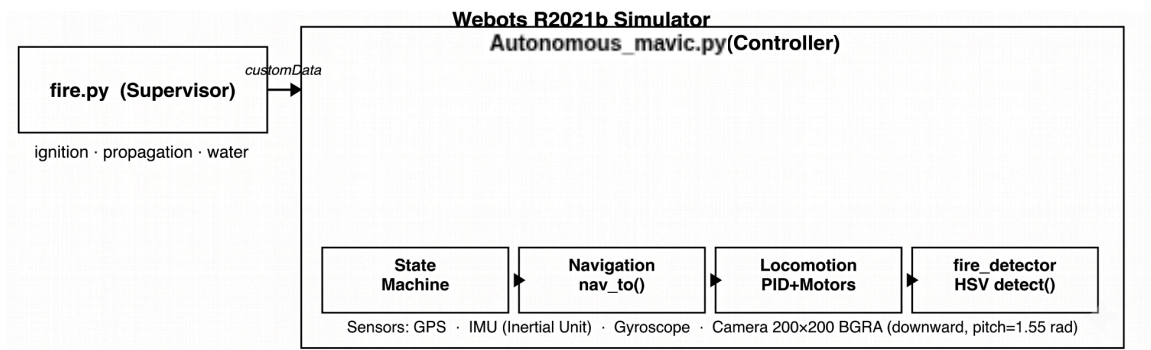


Figure 1: System architecture. Arrows indicate data flow. `customData` channel carries water-drop quantity from controller to supervisor.

2. Key Algorithms

2.1 Locomotion

Stable flight is achieved using proportional control. The IMU and gyroscope provide roll, pitch and yaw information while the GPS provides altitude. A proportional altitude controller keeps the drone close to the target altitude, while roll and pitch gains stabilise the aircraft. Finally, motor mixing converts roll, pitch, yaw and vertical corrections into individual propeller speeds.

2.2 Patrol Navigation

The drone follows a list of patrol waypoints. It continuously calculates the direction to the next waypoint using $\arctan2()$, rotates towards it, then flies forward. Once it reaches the waypoint within the required precision, it automatically moves to the next one. Patrol is interrupted immediately whenever smoke is detected.

2.3 Smoke Detection

Each camera frame is converted from BGRA to RGB, rotated and flipped before being processed with OpenCV. The image is converted into HSV colour space because it separates colour from brightness and makes smoke easier to isolate. Smoke pixels are extracted using HSV limits of Lower=(0,0,90) and Upper=(172,100,255). The controller ignores very small detections and keeps only the largest contour with a radius greater than 3 pixels. After detection, the coordinates are swapped from (raw_x, raw_y) to (control_x, control_y) so that the controller's movements match the rotated image.

2.4 Visual Servo Navigation

Instead of estimating GPS coordinates for the fire, the latest controller directly tracks the smoke in the camera image. The horizontal image error controls yaw while the vertical error controls forward or backward motion. The drone first aligns itself horizontally, then moves until the smoke is centred. If the smoke disappears briefly, the controller performs a short recovery using the last known position before giving up and returning to patrol.

2.5 Fire Extinguishing

When the smoke remains centred for four consecutive control updates, the controller sends a water-drop command. After dropping water, smoke detection is paused for 15 seconds to avoid detecting the water cloud or residual smoke as a new fire.

2.6 State Machine

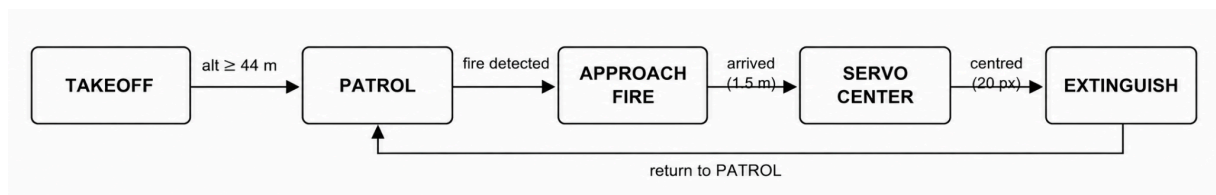


Figure 2 — Autonomous control state machine. Fire queue enables multi-fire handling without returning to PATROL between fires.

3. Challenges Faced and Solutions

Challenge	Solution
False smoke detections from vegetation.	I tuned the HSV limits using actual simulation frames until only smoke remained inside the detection mask.
Camera orientation did not match controller coordinates	I rotated and flipped the image, then swapped the detected coordinates so the drone turned in the expected direction
ROS 2 Humble PYTHONPATH conflict: System PYTHONPATH resolved 'from controller import Robot' to the ROS 2 controller package, producing an undefined symbol error.	Prefixed Webots python39 bindings and user site packages to PYTHONPATH in the launch command, overriding the ROS 2 entry.
Version conflicts. Project requires R2021b while my system had R2025a. R2021b bindings compiled for CPython 3.9, incompatible with my system Python 3.10.	Installed R2021b via .deb to overwrite R2025a. Installed Python 3.9.25. Webots Preferences set to python3.9.
Smoke scale too small at early detection	Detection threshold lowered. Drone patrols at 45 m altitude, maximising camera footprint over forest.
Repeated water drops at same location: After extinguishing, residual smoke immediately re-triggered detection at the hover position.	15 second cooldown after dropping water suppresses detection. On expiry, drone returns to PATROL to reacquire rather than retrying in place.

4. Performance Results

Results recorded on Linux Mint 21.3 (Ubuntu 22.04), Webots R2021b, running at approximately 0.2 times realtime speed due to my laptop's hardware constraints

Metric	Value	Notes
Takeoff to patrol altitude (45 m)	< 30 s of simulation time	Consistent across all runs
First fire detection (after ignition)	1 to 2 patrol passes	Depends on ignition tree location
Smoke-free false positives	1 to	Less false triggers after HSV tuning
Fire ratio over smoke	0.35–0.60	Depends on smoke growth
Fires detected per run	4	All visible fires detected
Fires extinguished per run	4	Successful

Total sim time to extinguish all fires	5.8 min	Includes water cooldown
--	---------	-------------------------

5. Environment and Dependencies

Component	Detail
Operating System	Linux Mint 21.3 (Ubuntu 22.04 base)
Simulator	Webots R2021b
Python	3.9
OpenCV	opencv-python 4.13.0.92
NumPy	2.0.2
Robot	Mavic 2 Pro (two)
Controller file	controllers/autonomous_mavic/autonomous_mavic.py